Time to move to unit tests to drive that logic. But where do the tests go? You have more design decisions to make.

There are two distinct responsibilities in blocking a user. There's the actual logic to determine whether a user can see a project, and there's the logic to redirect the user if the access checker fails. Let's test those responsibilities separately.

Let's deal with the access control first. That method is probably either Project#can_be_viewed_by?(user) or User#can_view?(project). The active-voice version in User seems more clear. Another option is the Pundit gem,[3] which puts the access-control policy in its own separate object, but I'm reluctant to bring in another non-test-related gem here. If the access code gets more complex, though, I'd move it to prevent the User class from getting too big.

Right now you're testing just the access logic, which means that all you need for each test is one user and one project:

security/02/spec/models/user_spec.rb

```
require "rails_helper"

RSpec.describe User, type: :model do
  let(:project) { create(:project) }
  let(:user) { create(:user) }

  it "cannot view a project it is not a part of" do
    expect(user.can_view?(project)).to be_falsy
  end

  it "can view a project it is a part of" do
    Role.create(user: user, project: project)
    expect(user.can_view?(project)).to be_truthy
  end
end
```

The two cases here are similar to the integration tests: either the user is a member of the project and can see it, or the user isn't a member of the project and can't see it.

The second test has one subtle point about how Rails handles associations. In the first test, where the user is not a member of the project, you technically don't need to save the user, project, and role to the database to run the test. In the second test you do. (I'm saving them to the database to keep the test cases similar, but you could get away with not saving them in the first test in order to have a faster test).

3. https://github.com/elabs/pundit